

20 Full Stack Development Project Case Studies

Technology Stack: PostgreSQL, ASP.NET Core Web API, Angular, Azure Services, Docker & Kubernetes (Phase 2)

1. Hospital Appointment & Patient Management System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Doctor Management
- Patient Registration
- Appointment Booking
- Prescription Management
- Lab Reports Upload
- Billing

Key Features:

- Push notifications for appointments
- Azure Blob Storage for reports
- Role-based access
- JWT Authentication

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage
- Azure Key Vault
- Azure Notification Hub

Student Deliverables:

- Requirement Document
- ER Diagram

- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Dockerization
- Kubernetes deployment
- AI-based symptom suggestions

2. Smart Inventory & Warehouse Management

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Inventory Tracking
- Supplier Management
- Purchase Orders
- Barcode Management
- Warehouse Transfers

Key Features:

- Low-stock alerts
- Multi-warehouse support
- Search & pagination

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Service Bus
- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Microservices
- Redis caching

3. HRMS & Employee Lifecycle Management

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Employee Management
- Attendance
- Leave
- Payroll
- Performance Reviews

Key Features:

- Push notifications
- Document uploads
- Role-based access

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Key Vault
- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Docker
- Kubernetes

4. Real Estate Property & Rental Platform

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Property Listings
- Tenant Management
- Lease Tracking
- Rent Collection

Key Features:

- Image gallery
- Geo-location search
- Complaint management

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Containerization
- Search optimization

5. Online Examination & Assessment Platform

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Question Bank
- Exam Scheduling
- Auto Evaluation
- Certificates

Key Features:

- Timed exams
- Analytics dashboard
- Real-time notifications

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Notification Hub

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Scalable Kubernetes deployment

6. Restaurant Order & Kitchen Management

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Order Management
- Menu Management
- Kitchen Queue
- Billing

Key Features:

- SignalR live updates
- Kitchen notifications
- QR ordering

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Service Bus

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Dockerized services

7. Insurance Claim Processing System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Policy Management
- Claims Submission
- Document Verification
- Approvals

Key Features:

- Workflow approvals
- Audit logs
- Notifications

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage
- Azure Key Vault

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Microservices

8. Logistics & Shipment Tracking System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Shipment Booking
- Tracking
- Driver Management
- Delivery Confirmation

Key Features:

- GPS tracking simulation
- Push updates

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Maps
- Azure Service Bus

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Kubernetes

9. E-Commerce Marketplace Platform

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Products
- Cart
- Orders
- Reviews
- Vendor Management

Key Features:

- Wishlist
- Discount engine
- Push notifications

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure CDN
- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Microservices
- Docker

10. School ERP Management System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Students
- Attendance
- Fees
- Homework
- Timetable

Key Features:

- Parent portal
- Notifications
- Report generation

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Containerization

11. Travel & Tour Package Management

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Packages
- Bookings
- Itinerary
- Reviews

Key Features:

- Travel document upload
- Seasonal pricing

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Microservices

12. Event Management & Ticketing Platform

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Event Creation
- Ticket Booking
- Seat Allocation
- QR Entry

Key Features:

- Real-time availability
- Email queue

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Service Bus

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Scalable deployment

13. Fleet & Vehicle Maintenance System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Vehicle Tracking
- Fuel Logs
- Maintenance Scheduling

Key Features:

- Predictive maintenance
- Reminder alerts

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Notification Hub

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Docker & Kubernetes

14. Complaint & Grievance Redressal Portal

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Complaint Registration
- Escalation Workflow
- Tracking

Key Features:

- SLA tracking
- Notifications
- Analytics

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Service Bus

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Microservices

15. Manufacturing Production Monitoring System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Production Planning
- Machine Monitoring
- Defect Tracking

Key Features:

- IoT simulation
- Production analytics

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure IoT simulation
- Azure Service Bus

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Kubernetes

16. Learning Management System (LMS)

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Courses
- Assignments
- Discussion Forum
- Quiz

Key Features:

- Progress tracking
- Video uploads

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Dockerized deployment

17. Hotel & Resort Operations Management

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Room Booking
- Housekeeping
- Billing
- Food Orders

Key Features:

- Guest feedback
- Availability tracking

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Notification Hub

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Containerization

18. Agriculture & Farm Monitoring Platform

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Crop Monitoring
- Worker Management
- Yield Reports

Key Features:

- Weather integration
- IoT simulation

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Service Bus

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Kubernetes

19. Digital Document Approval Workflow System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Document Upload
- Approval Workflow
- Version Tracking

Key Features:

- Audit logs
- Multi-level approvals

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Blob Storage
- Azure Key Vault

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Microservices

20. Smart Society / Apartment Management System

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Residents
- Visitors
- Maintenance Billing
- Facility Booking

Key Features:

- QR visitor entry
- Push announcements

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Notification Hub

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Docker & Kubernetes

